TASK C · L2 SUPPORT ENGINEER ASSESSMENT

dxFeed demo feed assessment

Data consistency, behaviour under load, operational risk, an incident drill and three product improvements

Scope	dxFeed demo REST (events.json) and SSE (eventSource), Radar web tool
Instruments	AAPL (US stock), EUR/USD (FX), TSLA (high volatility), /ESZ26:XCME (futures control)
When	Friday 16 October 2026, 11:10–14:00 UTC (pre-market and US regular hours)
Evidence	taskC_assessment/evidence/. Every number below can be recomputed offline
Status legend	Verified = reproduced from saved evidence · Hypothesis = plausible, unproven · Not done = needs live access

Irakli — L2 Support Engineer candidate · 18 October 2026

1. Summary

I spent a working day treating the dxFeed demo service the way an L2 engineer would treat a customer complaint about data quality: pull enough history to have a statistically useful sample, rebuild every larger timeframe from the smaller one, compare against what the service serves, and then push the service with a polite, self-limiting load test. Everything was logged (URL, parameters, UTC time, latency, body hash), so any number in this report can be recomputed from disk without touching the demo again.

58,315	~900 s	~8,000	~3 req/s	0
candles analysed	feed delay on every symbol	silent retention cap per series	saturation throughput (market hours)	HTTP errors in 648 load requests

Headline. The five things I would tell a customer-facing team first:

1. **The feed is 15 minutes delayed.** Quote age sits on 900 s for every instrument on REST, SSE and Radar. Any freshness alert must be written relative to that floor.
2. **History is silently capped at about 8,000 candles per symbol and period.** Older windows answer status OK with zero candles, which looks exactly like a market holiday.
3. **Hourly US-stock volume disagrees with the 5-minute sum, almost entirely outside regular hours.** AAPL 130 extended-hours hours plus 1 regular-hours hour out of 208; TSLA 118 + 2. Which series is the reference is not yet decided.
4. **EUR/USD prints 50 one-minute candles outside its session.** 7 after the Friday close and 43 before the Sunday reopen. An earlier count of 214 included the legitimate Sunday reopen and is withdrawn.
5. **Under load the service queues instead of refusing.** Throughput flattens at about 1.6 req/s pre-market and about 3 req/s in market hours; latency climbs into tens of seconds and the service never sends 429 or Retry-After.

What this report does not claim

- No root cause for the volume split. The TimeAndSale feed I needed to test it does not work on the demo.
- No production capacity number. This was one client, one IP, on a shared free demo.
- No post-weekend re-check. Pass 2 and the Monday reopen were not run.

2. How I approached it

Before writing any analysis code I spent the first hour on reconnaissance. The REST help page redirects from tools.dxfeed.com to demo.dxfeed.com and documents two resources: events.json for snapshots and eventSource for Server-Sent Events. The candle symbol grammar comes from the public CandleSymbol Javadoc, for example AAPL{=5m} or EUR/USD{=m,price=bid}. Three practical traps turned up immediately: curl strips the braces unless called with -g; an unknown symbol does not fail fast but

waits for a server timeout and answers HTTP 200 with status TIMED_OUT; and EUR/USD only has candles when price=bid is requested.

Instrument choice

Role	Instrument	Why
US stock	AAPL	Most liquid US equity; the demo serves it with extended hours, so regular and extended sessions can both be studied.
FX pair	EUR/USD	Most liquid pair. Only the bid-price candle exists on the demo (an edge case in itself).
High volatility	TSLA	The brief suggested crypto, but every crypto symbol I tried returned TIMED_OUT. TSLA is the most volatile liquid name served: realised 1-minute volatility 5.9 bp vs AAPL 5.1 bp.
Control	/ESZ26:XCME	CME E-mini S&P futures, nearly 24 hours, used as a clean session reference for the weekend question.

Pipeline

1. **Pull history in chunks.** fetch_candles.py made 96 requests in pass 1 (11:34–11:37Z): 1m and 5m from 2 October, 1h from 1 January so that all seven NYSE holidays of 2026 and the March DST change are covered.
2. **Validate every series.** validate_candles.py checks duplicates, ordering, alignment, gaps inside the expected session, candles outside the session, frozen prices, NaNs, the forming last candle, holidays (from my Task A calendar) and DST.
3. **Rebuild and compare.** 5m is rebuilt from 1m and 1h from 5m (first open, max high, min low, last close, summed volume) and compared candle by candle with what the service serves.
4. **Cross-check other surfaces.** sse_compare.py streams the same symbols while polling REST; Radar was read by hand.
5. **Load test.** load_test.py ramps 1 → 5 → 10 → 25 → 50 parallel workers, with automatic stops; analyze_load.py turns the log into tables and plots.
6. **Re-check offline.** After an external review questioned several numbers, offline_recheck.py recomputed every headline figure from disk. Four statements were corrected; they are flagged where they appear.

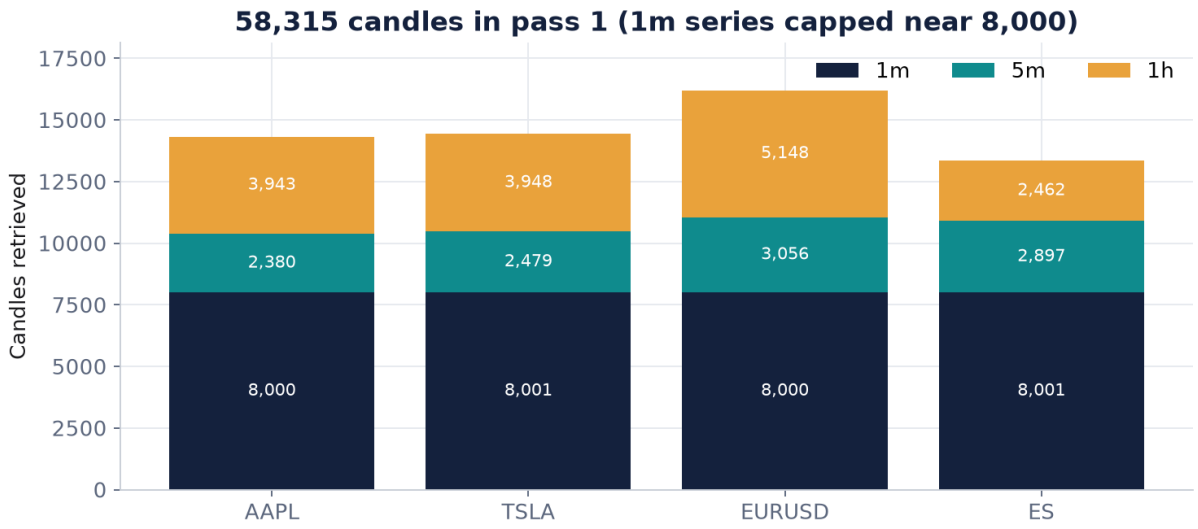


Figure 1. *Candles retrieved in pass 1 per instrument and period. The 1-minute series all stop at about 8,000, which is the retention cap in finding 3.2.*

3. Part 1: data consistency and integrity

3.1 What is right

I want to start with what works. Across the 12 series (58,315 candles) there are no timestamps off the interval grid, no out-of-order responses, no NaN prices and no candle whose open or close falls outside its own high–low range. Chunked downloads produce 21 boundary duplicates because toTime is inclusive, and all 21 carry identical values, so de-duplication is safe. Equities have zero hourly candles on every 2026 NYSE holiday so far, exactly matching the calendar the Task A scraper produced; EUR/USD trades straight through them, as it should.

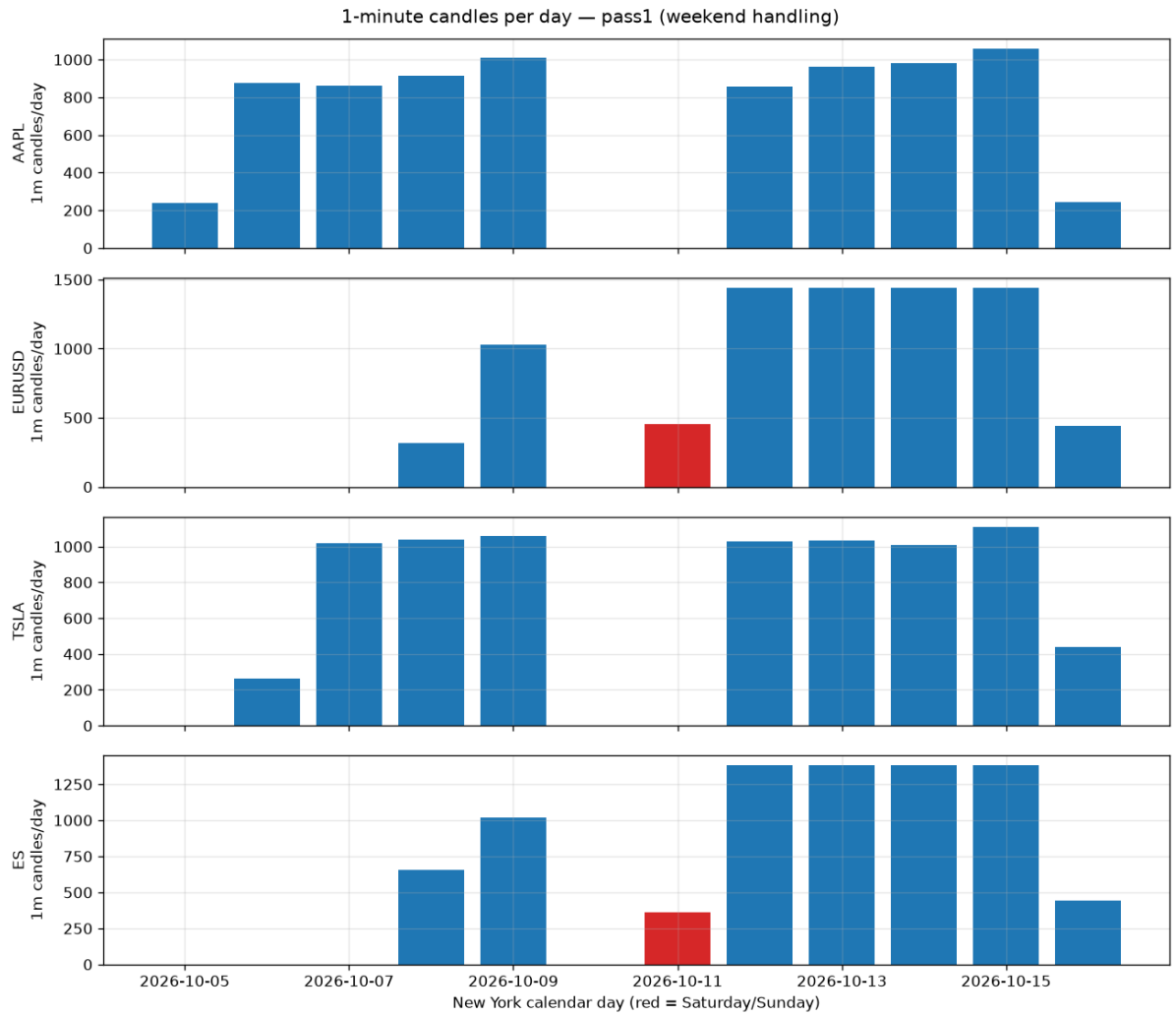


Figure 2. *One-minute candles per calendar day. Holidays and weekends appear as gaps, not as bad timestamps.*

Session model. The session model needed one correction. The first hourly bar of the New York day is at 00:00 ET, not 04:00 ET: the demo serves AAPL and TSLA every hour from 00:00 to 19:59 ET, with an overnight tail that a classic 04:00–20:00 pre-market model would call out-of-session. In this report regular hours (RTH) means Monday–Friday 09:30–16:00 ET and extended hours (ETH) means every other served candle. DST is handled correctly: the UTC hour of the first bar moves by one in March, the New York hour does not.

3.2 Silent retention of about 8,000 candles (verified)

A two-week request for AAPL 1-minute candles returns the newest 8,000 (TSLA and ES 8,001 after merging) and simply stops, with status OK. The same happens for 5-minute data: EUR/USD from 1 August returns 8,000 candles starting 9 September. The test that settled it was a window entirely before that edge: it also answers OK, with an empty 15-byte body. That rules out a response-size limit and confirms a retention limit, and it means a request for older data is indistinguishable from a holiday.

Why it matters

A backtest asking for 'the last two weeks of 1-minute bars' silently loses the first week, and nothing in the response lets the client alert on it.

3.3 Hourly volume does not equal the 5-minute sum (verified pattern, cause unknown)

This is the finding I spent most time on. Rebuilding AAPL hourly candles from 5-minute candles, 131 of 208 hours disagree on volume. 130 of them are outside regular hours and in every one the served hourly bar has more volume than the sum of its 5-minute bars, by up to 99 %. On 7 October at 17:00 ET the hourly bar reports 743,918 shares against 6,422 from the 5-minute bars. TSLA shows the same shape: 118 extended-hours hours plus 2 regular-hours ones. 5-minute and 1-minute candles agree on every complete bucket, so the split is specifically between the hourly record and the intraday records.

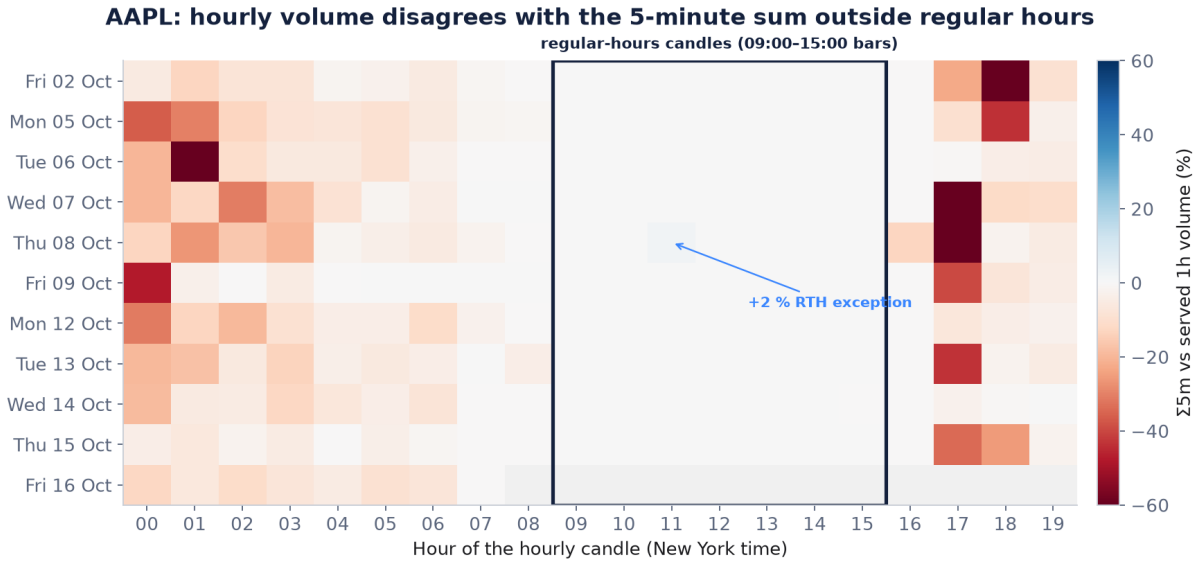


Figure 3. *AAPL: percentage difference between the rebuilt (Σ5m) and served hourly volume, by New York hour and day. Red = the served hourly bar carries more volume. Regular hours (boxed) are white except one hour.*

The single regular-hours exception is the most informative row. On 8 October at 11:00 ET the rebuilt volume is about 2 % higher than the served hourly bar, so the sign is reversed, while open, high, low and close are identical and the hour is complete (12 of 12 five-minute bars). The two TSLA regular-hours exceptions fall on the same day and overlapping hours, which points at a market- or venue-wide event rather than something about AAPL. Re-fetching both hourly bars 11.5 minutes apart returned identical values, so these are not candles still being revised.

Which series is the reference?

I only hold one daily candle for AAPL (14 October). Its volume is 50,061,053. The served hourly bars sum to 49,994,234 (-0.13 %), the 1-minute bars to 49,983,889 (-0.15 %). The hourly series is closer, and on every other full day the hourly total is at least the 1-minute total. That suggests the intraday series is the one missing prints, but it is a single data point. I therefore do not advise customers to rebuild extended-hours analytics from 1-minute data; what I would tell a client today is 'we are confirming which series is the reference'.

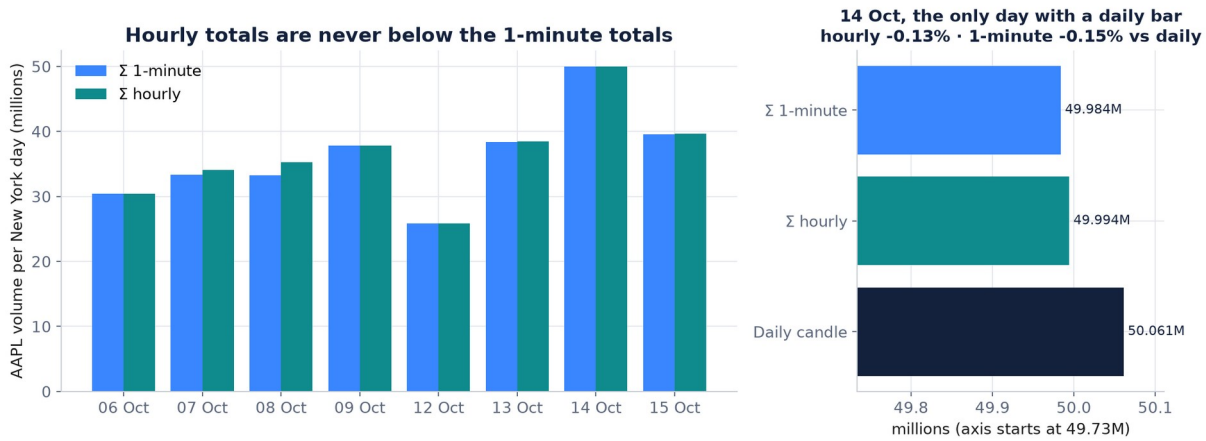


Figure 4. *Left: AAPL volume per New York day, summed from 1-minute vs hourly candles. Right: the only day with a daily candle; note the axis starts near 49.7 million to make the 0.1 % differences visible.*

Hypothesis	For	Against	Status
Per-period aggregators apply different inclusion rules for extended-hours sale conditions	Systematic ETH pattern; separate Trade.<period> records in the Javadoc	Does not explain the regular-hours exception's opposite sign	Hypothesis
Late prints or corrections land in one period's record only	Opposite-sign RTH exception; AAPL and TSLA on the same day	Bars were stable on re-fetch; prices identical	Hypothesis
Client-side summation error	—	1m ≡ 5m on volume and count; bid + ask = volume on both sides	Rejected

What would settle it. The test that would settle it is reading sale conditions from TimeAndSale for one mismatching hour. It is blocked: TimeAndSale and Trade history return TIMED_OUT or empty on the demo. Checking SPY and MSFT for 8 October 11:00 ET, and re-requesting with tho=true, both need live calls and were not done.

3.4 Price fields are almost, not perfectly, consistent (verified; corrected)

An earlier draft said OHLC always agrees across timeframes. It does not. Six hourly candles differ from their 5-minute rebuild in at least one price field. Three are the still-forming 11:00Z bar at fetch time, which is expected. Three are complete hours: AAPL on 2 October 21:00Z (close 333.35 rebuilt vs 333.3015 served) and TSLA on 8 October 15:00Z and 16:00Z, the same regular-hours block as the volume exception. The differences are tiny, at most 0.023 %, but they are real.

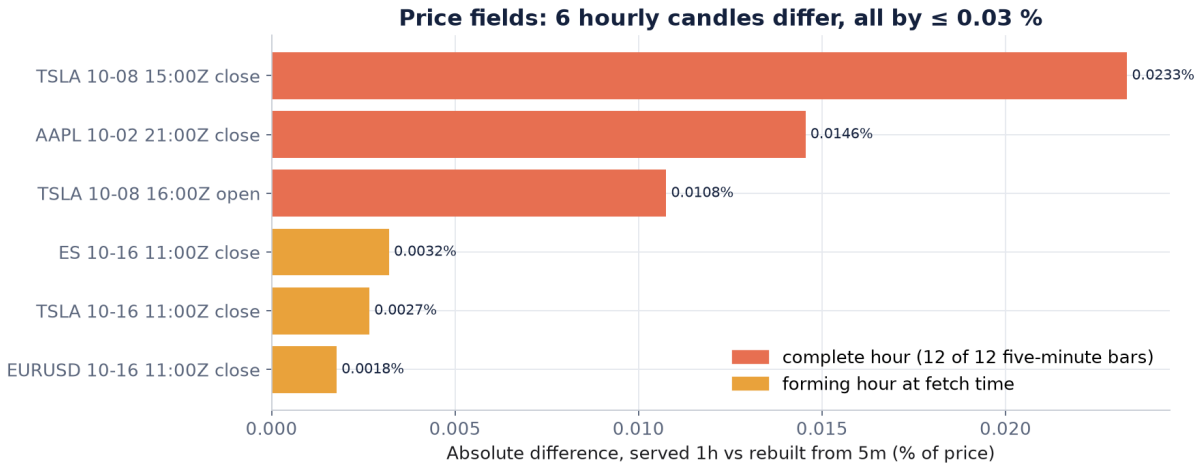


Figure 5. *Every price-field difference between served hourly candles and the 5-minute rebuild.*

3.5 EUR/USD prints outside its session (verified; corrected from 214 to 50)

FX trades from Sunday 17:00 ET to Friday 17:00 ET. My first draft tagged candles by UTC weekday and reported 214 'weekend' candles. 170 of those are Sunday 17:00–19:59 ET, which is the normal reopen and not an anomaly at all. Counting against the actual FX session leaves 50 one-minute candles (39 at 5 minutes): 7 just after the Friday close, none on Saturday New York time, and 43 sparse prints on Sunday before the reopen: a single tick at 09:20 ET, then a thin ramp from 14:00 ET. The ES futures control has zero candles outside its session, so the method itself is sound.

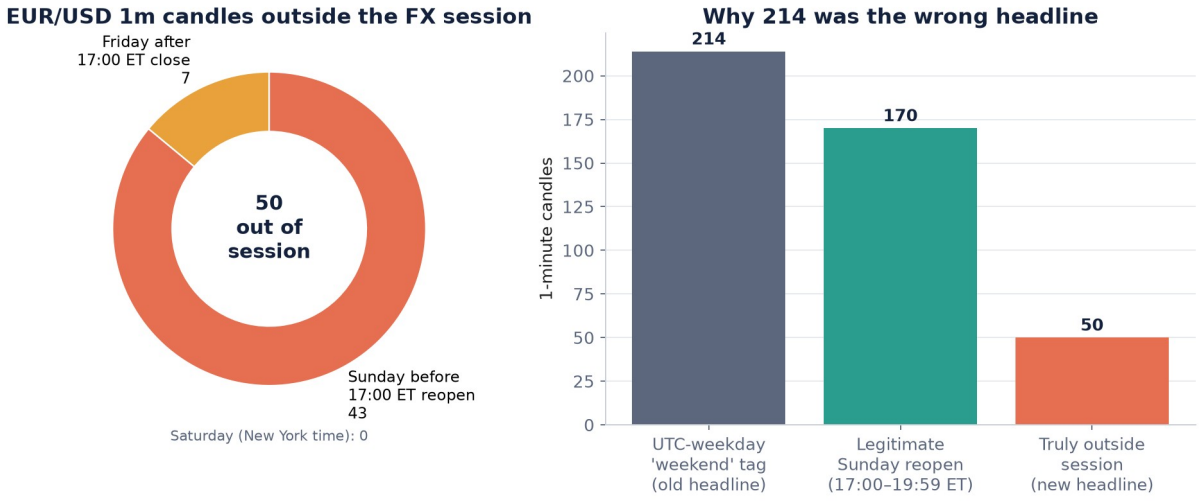


Figure 6. *Left: the 50 out-of-session EUR/USD candles by bucket. Right: how the UTC-weekday tag inflated the count.*

Two explanations fit: the liquidity provider publishes indicative quotes while the market is shut, or a heartbeat re-stamps a stale quote. A weekend quote logger was set up to tell them apart (does the price move or only the timestamp?), but it stopped on Saturday afternoon, so this stays open. For customers the risk is session logic: 'last candle is fresh' would call FX open on Sunday morning.

3.6 Smaller edge cases customers trip over (verified)

Edge case	What happens	Client impact
FX default candle	EUR/USD{=1m} → OK, no candles; only price=bid works; volume is a tick count	Silent empty result; meaningless FX 'volume'
Daily candle timestamp	Stamped 00:00 UTC of the trading date = 20:00 ET the day before	Every daily bar labelled with the previous date
Inclusive toTime	A 60-minute window returns 61 one-minute candles	Duplicates at chunk boundaries (21 found, all identical)
Unknown symbol	HTTP 200 + TIMED_OUT after ~3.3 s idle, ~29 s under load	Looks like success; slows every mixed request
Forming candle	Returned without any flag (eventFlags = 0)	Clients store a candle that later changes
SSE inside the delay window	NaN control candles dated 2038-01-19	Parsers that ignore flags store garbage
Radar vs REST universe	Radar shows NVDA/TSM; REST times them out	Support confusion: 'it works in Radar'

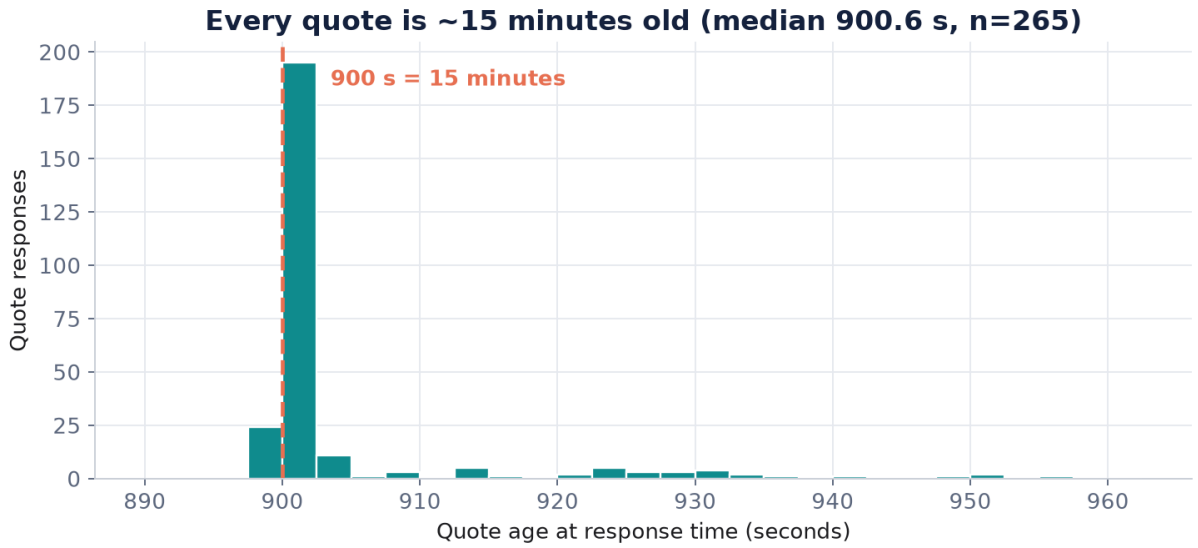


Figure 7. *Quote age at response time across both load runs: a hard floor at 900 seconds.*

4. Part 2: behaviour under load

The load test was designed to be polite and self-limiting: one aiohttp session, each worker holding at most one request, 0.25 s think time, 30-second steps at 1, 5, 10, 25 and 50 workers with 10-second pauses, and hard stops on the first 429, three consecutive server errors, a step error rate above 20 %, a step median above 8 seconds, or a 3,500-request budget. Each worker rotates through three request types: a 13-symbol Quote snapshot, a 'last 60 minutes' candle request and a fixed historical window whose body must be byte-identical every time. I ran it pre-market (11:52Z, stopped at 25 workers) and again in US regular hours (13:47Z, reached 50 workers).

Units

All latencies in this section are in seconds, not milliseconds.

Workers	Requests	Throughput (req/s)	p50 (s)	p95 (s)	p99 (s)	HTTP errors
1	39	1.31	0.45	0.81	2.37	0
5	99	3.18	1.14	2.30	3.96	0
10	104	3.19	2.54	4.22	6.31	0
25	124	3.02	5.28	12.17	14.19	0
50	138	2.78	8.28	26.55	28.24	0

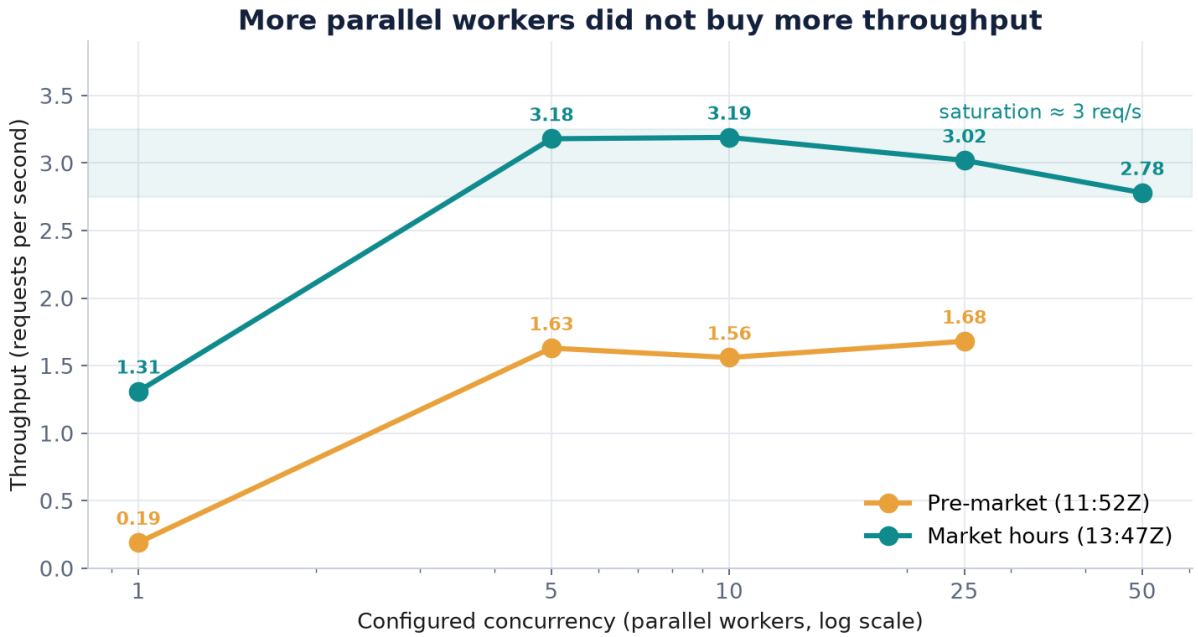


Figure 8. *Throughput measured through request completion. Pre-market plateaus at 1.63–1.68 req/s, market hours at about 3 req/s and then declines.*

The picture is a service that saturates early and then queues. Going from 5 to 50 workers in market hours did not add throughput. It fell slightly, from 3.18 to 2.78 req/s, while median latency rose from 1.1 to 8.3 seconds and p95 to 26.6 seconds. Nothing ever failed: 648 load requests, zero HTTP errors, no 429, no Retry-After header. A naive client that adds parallelism when things get slow only makes its own latency worse, and the service never tells it to back off.

Correction. An earlier version reported throughput 'rising to 4.6 req/s'. That figure divided requests by the time between the first and last request start and so ignored requests still in flight at the end of a step. Measured through completion the curve is flat. The corrected figures are used everywhere.

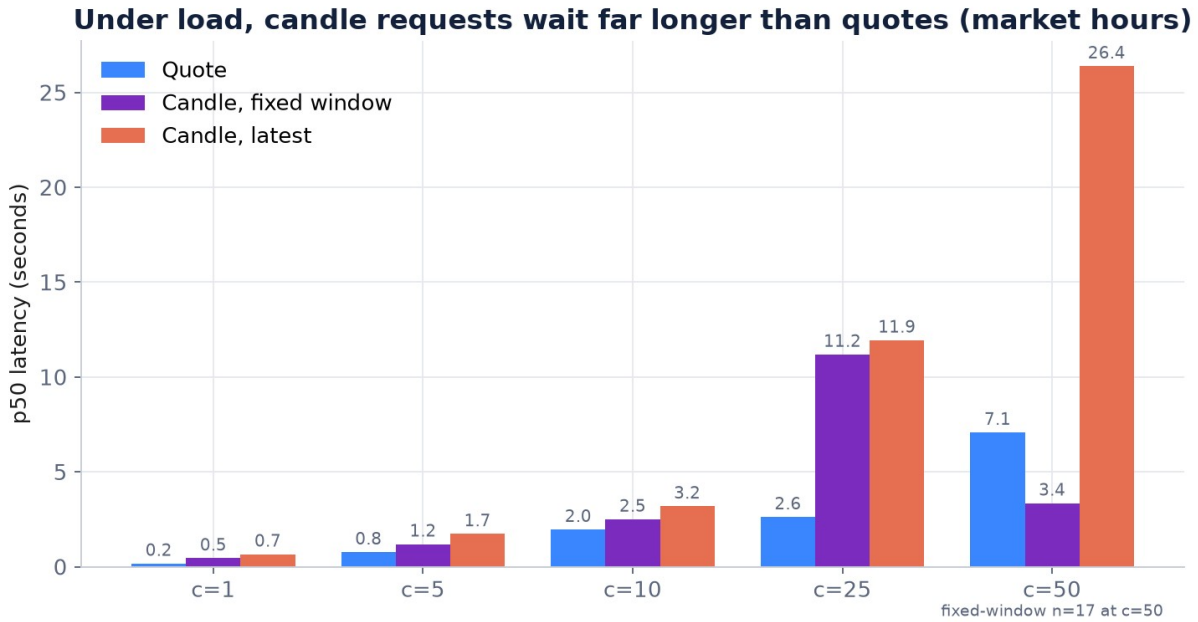


Figure 9. *Median latency by request type in market hours. Once saturated, candle requests wait far longer than quotes.*

Queueing is not uniform. At 25 workers quotes still came back in 2.6 seconds while both candle request types took 11–12 seconds, which fits per-request cost or separate queues rather than one FIFO. The TIMED_OUT path stretches the same way: a request for an unserved symbol (AUD/USD bid candles, part of the mix by design) took 3.3 seconds alone and 29 seconds at 50 workers, because the queue wait is added before the server-side timeout starts.

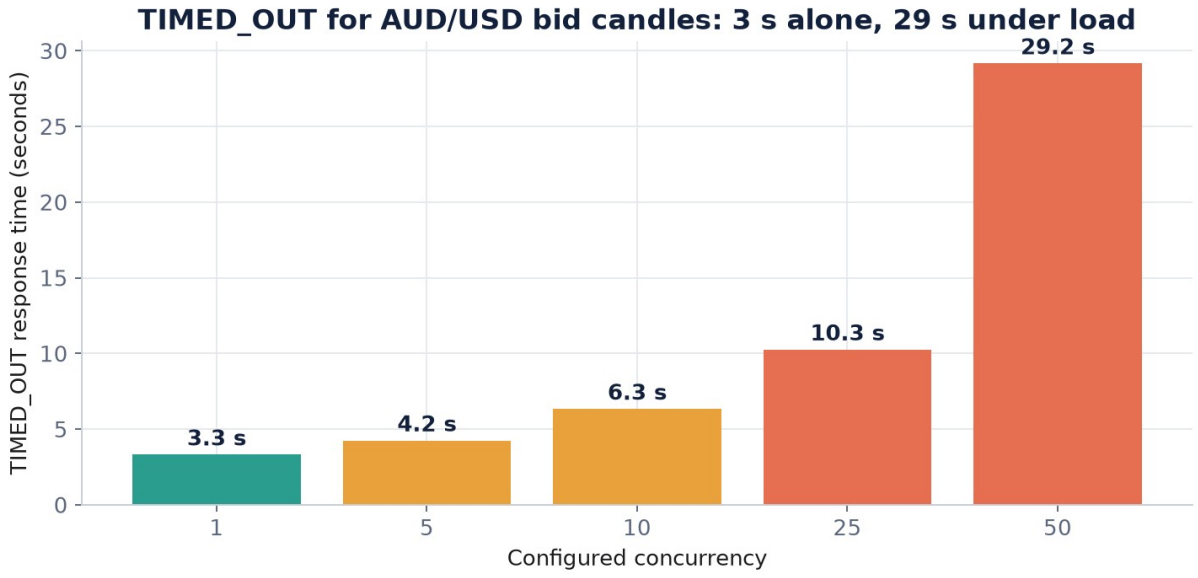


Figure 10. *TIMED_OUT response time versus concurrency, market hours.*

Is the limit on my side?

Little's law (in-flight = throughput × latency) on the request logs shows that at 5 and 10 workers the average number of requests in flight equals the configured concurrency, so my workers really were all waiting on the server. The aiohttp connection pool allows 52 connections, more than the 50 workers I ever ran, so the client pool could never have been the bottleneck. What I cannot say is whether the cap is a per-IP throttle or shared capacity used by other demo users; separating those needs a second client on a different IP running at the same time, which I did not run.

Consistency, caching and freshness under load

- **Consistency:** the fixed historical window returned byte-identical bodies on every repeat (AAPL 66/66, TSLA 62/62).
- **Caching:** no Cache-Control, ETag or Age headers, and repeats were not faster than the first call, so there is no sign of an HTTP cache.
- **Freshness:** quote age stayed at about 900 seconds at every step: load changes latency, not freshness.
- **Censoring:** percentiles use completed requests only, so at 25 and 50 workers they understate the true tail.
- **Streaming:** SSE was ahead of REST by a median 0.26 s in market hours; with a 5-second REST poll that is a lower bound, not a precise lag.

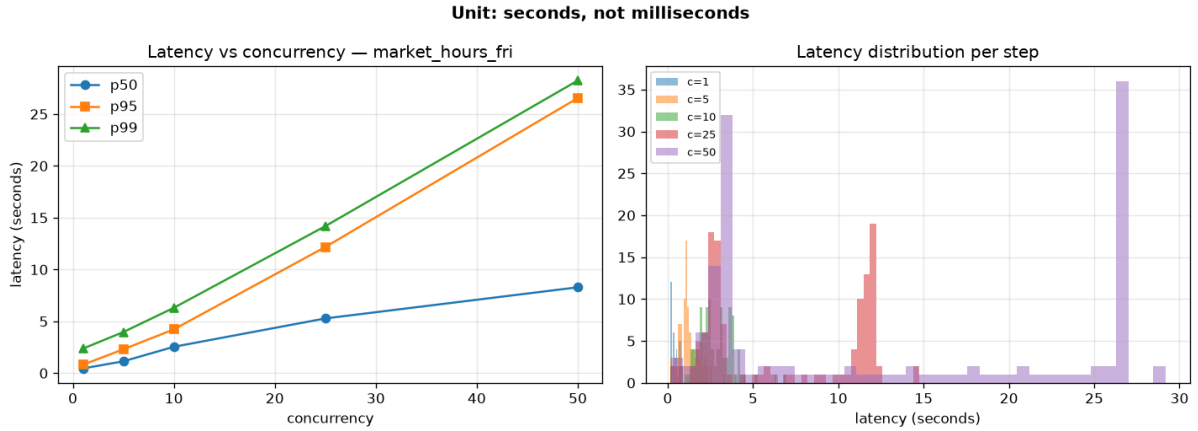


Figure 11. *Latency percentiles per step, market-hours run (seconds).*

Monitoring and SLOs I would set

Signal	Objective (demo service)	Escalate when
Availability (HTTP 200, parsable JSON)	99.5 % per month	Error rate > 1 % for 5 min or any 5xx burst
Latency, single quote or ≤ 1 day of candles	p95 ≤ 2 s over 5 min	p95 > 5 s for 5 min at concurrency ≤ 2
Freshness, liquid symbol in session	Age ≤ 900 s + 120 s for 99 % of minutes	Age > 1,200 s for 3 min while peers are fresh
Candle completeness, AAPL 1m in RTH	≥ 99.5 % of minutes within 20 min	< 98 % for an hour
Cross-timeframe price, complete buckets	≥ 99.9 % within 0.03 %	Any larger gap
TIMED_OUT duration	≈ 3.3 s baseline	Stretching with low client concurrency (queueing)

5. Part 3: operational risk review

I ranked risks by how often I actually saw them and by how much damage a silent failure does compared with a loud one. Silent and decision-affecting ranks above loud and cosmetic.

#	Risk	Evidence	Likelihood	Impact	Mitigation
1	Silent history cap (~8,000, status OK)	Finding 3.2	High	High	Truncation flag or documentation; client-side count check
2	Timeframe-dependent volume (mostly ETH)	Finding 3.3	High	Medium–high	Nightly 1h-vs-1m probe; publish aggregation rules
3	Saturation and queueing, no 429	Part 2	High under parallel use	Medium	Server: 429 + Retry-After; client: cap concurrency ≤ 5
4	Staleness indistinguishable from 'closed'	900 s delay, no freshness metadata	Always	High for real-time use	serverTime / delay fields; age-vs-calendar alert
5	Symbol and asset-class quirks	TIMED_OUT on 200; FX bid-only	Medium	Medium	Publish symbol list; distinct UNKNOWN_SYMBOL status
6	Out-of-session FX prints	Finding 3.5 (50 candles)	Every weekend	Low–medium	Session-calendar filter, not UTC weekday
7	Monitoring blind spot	OK on partial results; no rate-limit headers	High	Medium	Synthetic probes per asset class

6. Part 4: incident simulation INC-SIM-01

This is a simulation

The scenario and its timestamps are invented; every measurement quoted is real and reproducible from the evidence folder.

Scenario. A quant client's nightly reconciliation flags AAPL hourly volume for 18:00 ET on 2 October as 845,299 against 13,744 from their own 1-minute roll-up and opens a P1 'volume data corrupted'. Within fifteen minutes L2 reproduces it from public endpoints, counts 130 extended-hours mismatches and one regular-hours one, confirms 5-minute equals 1-minute, checks the daily total and escalates to the aggregation team with the CSV and one question: which series is the reference?

Time (UTC)	Event
Thu 20:30	Client reconciliation flags the 18:00 ET hour
21:05	P1 ticket opened by the client
21:15	L2 reproduces on demo REST: −98.4 %; 130 ETH + 1 RTH mismatches
21:40	Same pattern on TSLA; daily total checked (hourly closer)
22:00	Escalation to aggregation on-call with evidence and the reference-series question
22:30	First client update: facts, no cause, next-update time
Fri 09:00	Engineering still confirming the rule; RTH exception under investigation

Client message (external)

Thank you for the detailed report. We reproduced the difference between hourly candle volume and your 1-minute roll-up for AAPL on 2 October. It affects the volume field of hourly candles for US stocks, almost

entirely outside regular trading hours; prices agree apart from a few differences below 0.03 %. Our data engineering team is confirming the aggregation rule and which series is the reference. Until then please do not switch your extended-hours volume source on our account; if you need a number today, reconcile against the hourly and daily totals and tell us which one you use. Next update by 10:00 UTC tomorrow.

Why the message is cautious. No workaround is offered. tho=true (regular-hours-only candles) is a candidate but was never tested, and rebuilding hourly bars from 1-minute data would give the lower of the two numbers.

Post-mortem outline

1. Summary and severity; duration from first client report to published explanation.
2. Impact: symbol class, periods, fields, number of clients requesting 1h US-equity candles; no price impact beyond ≤ 0.03 %.
3. Timeline: detection by the client, reproduction, escalation, communication, decision.
4. Root cause: the confirmed inclusion rule, why it differs per period, why it was undocumented.
5. Went well: reproducible in ten minutes from public endpoints; message without speculation.
6. Went badly: found by the client; no cross-timeframe probe; status OK gives no hint.
7. Actions: nightly 1h-vs-1m probe (monitoring), aggregation rules in docs (docs), single rule or per-period attribute (aggregation), known-issues page (support).

7. Part 5: three product improvements

Area	Problem (evidence)	Proposal	Complexity	Business impact
Technical	Silent cap, unflagged forming candle, no delay in payload, slow TIMED_OUT for unknown symbols	Response envelope with truncated, limit, serverTime, delaySeconds, lastEventTime; flag the forming candle; immediate UNKNOWN_SYMBOL	Medium. Additive JSON, but it touches the web tier and the feed layer	Removes the two most likely 'your data is wrong' tickets
Monitoring	Nothing server-side would catch findings 3.2–3.5 or the queueing	One synthetic probe per asset class: age vs calendar, completeness, 1h-vs-1m, p95 at c=1 and 5, TIMED_OUT ratio	Small to medium. The scripts exist; wiring and a session calendar remain	Client-reported incidents become internally detected
Documentation	Candle syntax, price=bid, delay, symbol list, cap, inclusive toTime, UTC daily stamp each needed experiments	One 'demo service facts' page with worked examples and Radar-vs-REST differences	Small. This report is the draft	Fewer first-contact tickets, faster evaluations

8. What was not done, and why

Item	Status	Reason / next step
Pass 2 re-pull after the weekend	Not done	Run fetch_candles.py --pass pass2 after Monday 15:30Z
Weekend logger analysis	Partial	Logger stopped Saturday 14:28Z; Sunday reopen not covered
TimeAndSale sale-condition test	Blocked	TimeAndSale/Trade history TIMED_OUT or empty on demo REST
SPY/MSFT check of the 8 Oct RTH hour	Not done	Needs live API calls
tho=true re-request	Not done	Needs live API calls
Burst and batching tests	Not done	Ramp only; no one-shot burst or 13-in-1 batching comparison
Second client / IP	Not done	Needed to separate per-IP throttle from shared capacity
Production capacity	Not done	Out of scope: shared free demo

Appendix: reproduce offline

From deliverables/taskC_assessment/scripts with the dxfeed conda environment:

```
python validate_candles.py --pass pass1
python analyze_load.py --run market_hours_fri
python offline_recheck.py
python make_report_charts.py
python ../build_report.py
```

Evidence folder	Contents
evidence/candles/pass1/	Parsed CSVs and gzip-compressed raw responses, chunk log
evidence/validation/pass1/	report.json/md, per-candle aggregation CSVs
evidence/load/<run>/	requests.jsonl, steps.jsonl, analysis.md/json, littles_law.md
evidence/sse/, evidence/radar/	Streaming comparison and Radar screenshots
evidence/recheck/recheck.json	Every headline number recomputed after review